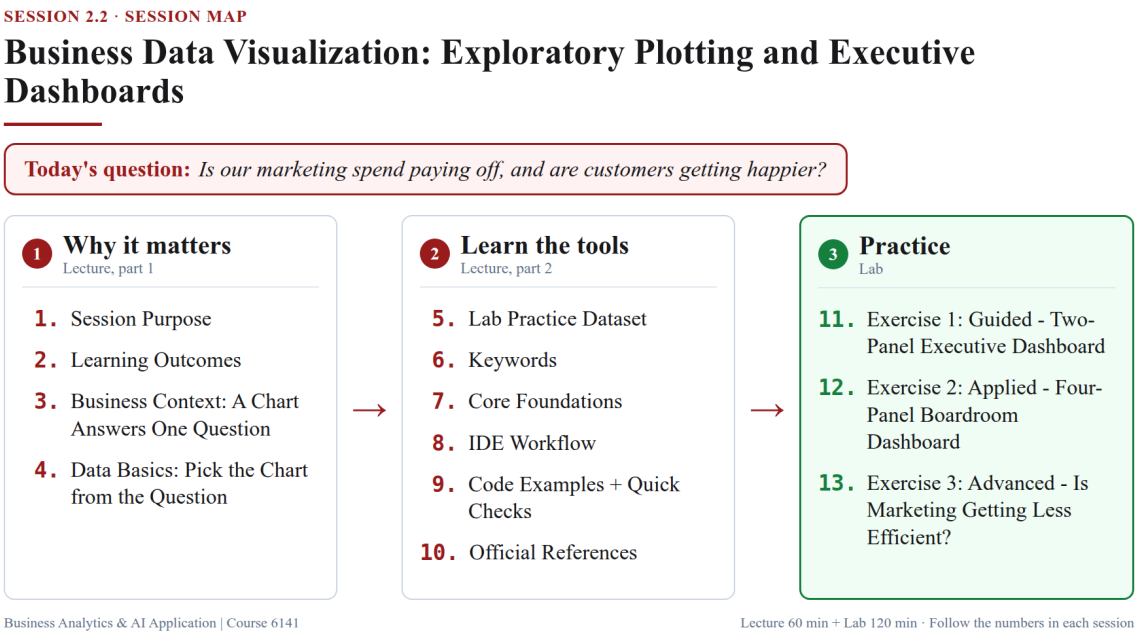
Session 2.2: Business Data Visualization: Exploratory Plotting and Executive Dashboards

Course 6141: Business Analytics & AI Application | Reading for Week 6



Schedule: Week 6 | In-person classes (1:2 Delivery Ratio: 60 Min Lecture + 120 Min Lab)

1. Session Purpose

- The marketing team keeps 12 months of results in one table: revenue, marketing spend, cost per new customer, and customer loyalty
- Executives often have only a minute or two per chart, and a table of 72 numbers hides the trend
- The CMO (Chief Marketing Officer) asks: is our marketing spend paying off, and are customers getting happier? Today, Python turns the table into charts that answer this at a glance

2. Expected Learning Outcomes (By the end of this session, you will be able to)

1. "Is revenue growing month by month?": draw a line chart with Matplotlib (Python's main charting library) using `fig, ax = plt.subplots()` (`fig` is the whole image; `ax` is one chart inside it)
2. "In which months did we spend the most on marketing?": draw a bar chart with Seaborn (a library built on Matplotlib for quick, clean statistical charts)

3. "Which numbers move together?": read a correlation heatmap (a colored grid of correlations, each from -1 to +1), and fix an `IndexError` in a 2x2 dashboard

3. Business Context: A Chart Answers One Question

- A table makes the reader hunt for the pattern; a chart shows it. In our table, revenue rises from `12.4` to `25.8` (USD millions), but the jump in November and December is easy to miss among 72 numbers.
- Five rules for charts that executives trust:
  - One message per chart, and the title states it, for example "Revenue More Than Doubled from Jan to Dec"
  - Label every axis with its unit, such as "USD M" (millions of U.S. Dollars)
  - Bar charts start at zero; a cut-off axis makes small differences look huge
  - No 3D effects or decoration; every mark on the chart should carry data
  - Use colors that still work in black-and-white print and for color-blind readers

4. Data Basics: Pick the Chart from the Question

Business question	Chart	Python
How does a number change over time?	Line chart	`ax.plot()`
Which category is biggest?	Bar chart	`sns.barplot()` or `ax.bar()`
Do two numbers move together?	Scatter plot	`ax.scatter()`
Which of many numbers move together?	Correlation heatmap	`sns.heatmap()`

- **Correlation** measures how closely two columns move together, on a scale from -1 to +1:
  - `+1`: when one goes up, the other always goes up
  - `0`: no straight-line pattern
  - `-1`: when one goes up, the other always goes down
- Correlation is not causation. In our data, marketing spend and revenue have a correlation of `0.996`, but both also simply grew through the year. A high correlation alone does not prove that the spending caused the revenue.

5. Lab Practice Dataset

- File Link: `data/session_02_02_dataset.csv`
- Audited 12-row monthly performance dataset (fictional teaching data), one row per month:
  - `Month_Index`: Month number, 1 to 12.
  - `Month_Name`: Three-letter month name, `Jan` through `Dec`.
  - `Organic_Revenue_USD_M`: Organic revenue in millions of U.S. Dollars. "Organic" means revenue from the company's own operations, not from buying other companies.
  - `Paid_Marketing_Spend_USD_M`: Paid advertising spend in millions of U.S. Dollars.

  - `Customer_Acquisition_Cost_USD`: CAC (Customer Acquisition Cost): the average marketing cost to win one new customer, in U.S. Dollars.
  - `Net_Promoter_Score`: NPS (Net Promoter Score): a customer loyalty score from -100 to +100, based on how likely customers are to recommend the company. Our values range from `58` to `74`.

6. Keywords (Core Programming Concepts)

- `Matplotlib` and `Seaborn`:
  - What: Python charting libraries; Seaborn adds quick, clean statistical charts on top of Matplotlib
  - Why: they turn a DataFrame into a chart image with a few lines of code
- `Figure` and `Axes`:
  - What: the `Figure` is the whole image; each `Axes` is one chart inside it
  - Why: you draw on a specific `Axes`, so a dashboard can hold several charts
- `Line Chart`:
  - What: points joined in time order
  - Why: shows trends, such as monthly revenue
- `Bar Chart`:
  - What: one bar per category; the bar length is the value
  - Why: compares sizes, such as spend by month
- `Scatter Plot`:
  - What: one dot per row, with one column on each axis
  - Why: shows whether two numbers move together
- `Correlation Heatmap`:
  - What: a grid of correlations, colored from -1 to +1
  - Why: spots which of many numbers move together, in one picture

7. Core Programming & Computational Foundations

- Draw on an `Axes`, not on the screen:
  - `fig, ax = plt.subplots()` creates one figure with one chart area
  - Then call methods on `ax`: `ax.plot()`, `ax.set_title()`, `ax.set_ylabel()`
  - Why: every instruction goes to a named chart, which matters once a figure holds several charts
- Save the chart as a file:
  - `fig.savefig("data/chart.png", dpi=150, bbox_inches="tight")` writes an image file
  - `dpi` (dots per inch) sets the sharpness: 150 for screens, 300 for print
  - `bbox_inches="tight"` trims empty white borders
- Grids of charts:
  - `fig, axes = plt.subplots(2, 2)` creates 4 charts, and `axes` is a 2-row by 2-column table

  - Reach one chart with a row and a column number, both starting at `0`: `axes[0, 1]` is the top-right chart
  - `axes[2]` asks for a third row that does not exist, so Python raises an `IndexError`

- `plt.tight_layout()` spaces the charts so that titles and labels do not overlap.

8. Development Workflow in Google Antigravity IDE

- `[1]` File Explorer: create your script with your name and the session code (for example, `Reacher_S22_demo_1.py`), and check the dataset is in `data/`. Saved charts appear here as `.png` files; click one to open it.
- `[2]` Code Editor: write a few lines, run, repeat. Small steps keep errors small.
- `[3]` AI Agent Chat: on an error, capture the traceback (Python's error report) with `Win + Shift + S` (macOS: `Cmd + Shift + 4`), paste it with `Ctrl + V`, and ask why it failed.
- `[4]` Terminal: run `python` followed by your script name. Return code `0` means the script finished with no errors. First time only: `pip install matplotlib seaborn`.

9. Hands-On Coding Micro-Examples

- Every example has three parts: **1. Input**, **2. Process**, and **3. Output**. Name each file with your own name and the session code, for example `Reacher_S22_demo_1.py` (pattern: `YourName_S22_demo_1.py`).
- The examples save each chart as a `.png` file in `data/`. Open the file in Panel `[1]` to see the chart.
- *Micro-Example 1 (Line Chart: Revenue Trend):* `YourName_S22_demo_1.py`

```python
# 1. Input: load the monthly results
import pandas as pd
import matplotlib.pyplot as plt
df = pd.read_csv("data/session_02_02_dataset.csv")

# 2. Process: one figure with one chart, revenue by month
fig, ax = plt.subplots(figsize=(10, 5))
ax.plot(df["Month_Name"], df["Organic_Revenue_USD_M"],
        color="#991B1B", marker="o")
ax.set_title("Revenue More Than Doubled from Jan to Dec")
ax.set_ylabel("Organic Revenue (USD M)")

# 3. Output: save the chart as an image file
fig.savefig("data/revenue_trend.png", dpi=150, bbox_inches="tight")
print("Saved: data/revenue_trend.png")
```

  - `import matplotlib.pyplot as plt` loads Matplotlib's charting tools under the short name `plt`.
  - `figsize=(10, 5)` sets the image size in inches: 10 wide, 5 tall.
  - `ax.plot(x, y)` draws the line; `color` sets its color, and `marker="o"` puts a dot on each month.

- *Quick Check 1 (`ax.grid()`):*

  - New tool: `ax.grid(True, linestyle=":")` adds light dotted guide lines behind the chart. Why: readers can line up a point with its value on the axis.
  - Task: add one line at the end of part **2. Process**: `ax.____(True, linestyle=":")`
  - Check: open `data/revenue_trend.png`; dotted grid lines now sit behind the revenue line.

- *Micro-Example 2 (Bar Chart: Marketing Spend by Month):* `YourName_S22_demo_2.py`

```
# 1. Input: load the monthly results
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
df = pd.read_csv("data/session_02_02_dataset.csv")

# 2. Process: one bar per month
fig, ax = plt.subplots(figsize=(10, 4))
sns.barplot(data=df, x="Month_Name", y="Paid_Marketing_Spend_USD_M",
            color="#475569", ax=ax)
ax.set_title("Marketing Spend Peaks in November and December")
ax.set_xlabel("")
ax.set_ylabel("Paid Marketing Spend (USD M)")

# 3. Output: save the chart
fig.savefig("data/marketing_spend_bar.png", dpi=150, bbox_inches="tight")
print("Saved: data/marketing_spend_bar.png")
```

  - `sns.barplot(data=df, x=..., y=...)` reads the column names straight from the DataFrame; `ax=ax` tells Seaborn which chart to draw on.
  - `ax.set_xlabel("")` removes the x-axis label, because the month names already explain the axis.

- *Quick Check 2 (`.nlargest()`):*

  - New tool: `df.nlargest(3, "Column_Name")` returns the 3 rows with the largest values in that column. Why: confirms the chart's message with exact numbers.
  - Task: add one line to part **3. Output**: `print(df._____(3, "Paid_Marketing_Spend_USD_M")[["Month_Name", "Paid_Marketing_Spend_USD_M"]])`
  - Check: the Terminal lists `Dec` (`6.4`), `Nov` (`5.2`), and `Oct` (`4.1`).

- *Micro-Example 3 (Correlation Heatmap):* `YourName_S22_demo_3.py`

```
# 1. Input: load the monthly results
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
df = pd.read_csv("data/session_02_02_dataset.csv")

# 2. Process: correlation between every pair of 4 columns
cols = ["Organic_Revenue_USD_M", "Paid_Marketing_Spend_USD_M",
        "Customer_Acquisition_Cost_USD", "Net_Promoter_Score"]
corr = df[cols].corr().round(2)
```

```
fig, ax = plt.subplots(figsize=(8, 6))
sns.heatmap(corr, annot=True, cmap="RdBu_r", vmin=-1, vmax=1, ax=ax)
ax.set_title("All Four Measures Rose Together in 2024")

# 3. Output: print the numbers, then save the chart
print(corr)
fig.savefig("data/kpi_correlation_heatmap.png", dpi=150,
            bbox_inches="tight")
```

  - `.corr()` returns a table with the correlation of every pair of columns. Each column with itself is always `1.0`.
  - `annot=True` writes each number in its cell. `cmap="RdBu_r"` colors from blue (`-1`) through white (`0`) to red (`+1`); `vmin=-1, vmax=1` fixes that scale.
  - Result: every value is between `0.97` and `1.00`, because all four measures grew through the year. Remember: correlation is not causation.

- *Quick Check 3 (`.corr()` on two columns):*
  - New tool: `column_a.corr(column_b)` returns one correlation number for two columns. Why: answers a single question without a whole heatmap.
  - Task: add one line to part **3. Output**: `print(round(df["Paid_Marketing_Spend_USD_M"].__ __(df["Organic_Revenue_USD_M"]), 3))`
  - Check: the Terminal shows `0.996`.

10. Official Python Documentation & References

- Matplotlib Official Documentation: User Guide.
- Seaborn Official Documentation: Statistical Data Visualization.
- Edward Tufte: The Visual Display of Quantitative Information (Principles of Information Design).

11. Exercise 1: Guided Practice - Two-Panel Executive Dashboard

- *Step 1: Open Script:* In Google Antigravity IDE, create a new script named `YourName_S22_dashboard.py`, replacing `YourName` with your own name (for example, `Reacher_S22_dashboard.py`).
- *Step 2: Build Two Charts Side by Side:* Type the following code:

```
# 1. Input: load the monthly results
import pandas as pd
import matplotlib.pyplot as plt
df = pd.read_csv("data/session_02_02_dataset.csv")

# 2. Process: one figure, two charts side by side
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

# Left chart: revenue trend
ax1.plot(df["Month_Name"], df["Organic_Revenue_USD_M"],
```

```
        color="#991B1B", marker="s", linewidth=2)
ax1.set_title("Monthly Revenue (USD M)")
ax1.grid(True, linestyle=":")

# Right chart: cost per new customer vs customer loyalty
ax2.scatter(df["Customer_Acquisition_Cost_USD"],
            df["Net_Promoter_Score"], color="#1E293B", s=80)
ax2.set_title("Loyalty (NPS) vs Cost per New Customer (CAC)")
ax2.set_xlabel("CAC (USD)")
ax2.set_ylabel("Net Promoter Score")
ax2.grid(True, linestyle=":")

# 3. Output: save the dashboard
plt.tight_layout()
fig.savefig("data/executive_dashboard_2panel.png", dpi=200)
print("Dashboard saved: data/executive_dashboard_2panel.png")
```

  - `plt.subplots(1, 2)` makes 1 row with 2 charts, and `(ax1, ax2)` names them left and right.
  - `ax.scatter(x, y)` draws one dot per month; `s=80` sets the dot size.

- *Step 3: Inspect Output:* Open `data/executive_dashboard_2panel.png` in Panel `[1]` and check that titles and axis labels do not overlap.
- *Step 4: Execute in Terminal:* Run `python YourName_S22_dashboard.py` in Panel `[4]` and confirm return code `0`.
- *Success Check:* the image has 2 charts. The right chart's dots climb from bottom-left to top-right: months with a higher cost per new customer also had higher loyalty, which matches their correlation of `0.98`.

12. Exercise 2: Applied Practice - Four-Panel Boardroom Dashboard

- *Coding Challenge:* In `YourName_S22_practice.py`, build a 2x2 dashboard with 4 charts: revenue trend (line), marketing spend by month (bar), cost per new customer by month (line), and the correlation heatmap.
- *Task 1 (2x2 Layout):* Create the grid with `fig, axes = plt.subplots(2, 2, figsize=(16, 10))`. Draw each chart on its own position: `axes[0, 0]`, `axes[0, 1]`, `axes[1, 0]`, and `axes[1, 1]`. Pass `ax=axes[1, 1]` to `sns.heatmap()`.
- *Task 2 (Deliberate IndexError & AI Diagnosis):* Change one position to `axes[2].plot(...)`. Run the script and read `IndexError: index 2 is out of bounds for axis 0 with size 2`. Capture the traceback with `Win + Shift + S`, paste it into Panel `[3]` (AI Agent Chat) with `Ctrl + V`, and review the AI's explanation of row and column numbers.
- *Task 3 (Fix and Export):* Fix the position, call `plt.tight_layout()`, and save the dashboard to `data/boardroom_dashboard.png` with `dpi=300`.
- *Success Check:* `data/boardroom_dashboard.png` shows 4 charts with no overlapping titles, and the script ends with return code `0`.

13. Exercise 3: Advanced Practice - Is Marketing Getting Less Efficient?

- *Goal:* Answer the CMO's question with a new measure and one honest chart: does each marketing dollar still bring in as much revenue at the end of the year as at the start?
- *What you need:*
  - File: `data/session_02_02_dataset.csv` (described in **5. Lab Practice Dataset**).
  - Script name: `YourName_S22_challenge.py`.
  - Tools you already know: column math, `.round()`, `.mean()`, `.sort_values()`, a `for` loop with `if` / `else` (Session 1.2), `fig, ax = plt.subplots()`, and `fig.savefig()`.
- *Requirements (follow them in order):*
  1. Add `Revenue_per_Marketing_USD` = `Organic_Revenue_USD_M` / `Paid_Marketing_Spend_USD_M`, rounded to 2 decimal places. It shows how many dollars of revenue came with each dollar of paid marketing in that month.
  2. Compute the 12-month average of that column and round it to 2 decimal places.
  3. Build a list of 12 colors with a `for` loop: `"#991B1B"` (red) for months **below** the average, `"#94A3B8"` (grey) for all others.
  4. Draw a bar chart with `ax.bar()`: month names on the x-axis, `Revenue_per_Marketing_USD` as the bar heights, and your color list.
  5. Add a dashed horizontal line at the average, then a title that states your finding and a y-axis label with its unit ("USD of revenue per USD of marketing").
  6. Save the chart to `data/marketing_efficiency.png` with `dpi=150`.
  7. Print the average, then the best and the worst month with their values.
- *New tools for this exercise:*
  - `ax.bar(x_values, heights, color=color_list)`: Matplotlib's own bar chart. When `color` is a list, bar 1 gets color 1, bar 2 gets color 2, and so on.
  - `ax.axhline(y=5.08, linestyle="--", color="black")`: draws a horizontal line across the chart at height `y`, useful for marking an average or a target.
  - Building a list in a loop: start with `colors = []`, then `colors.append("#991B1B")` adds one item to the end.
- *Hints:*
  - Loop over the column itself: `for value in df["Revenue_per_Marketing_USD"]:`.
  - For the best and worst month, sort the DataFrame by the new column and read the first and last rows, for example with `.head(1)` and `.tail(1)`.
- *Self-Check (expected results):*
  - Average: `5.08`. Best month: `Jan` (`5.9`). Worst month: `Dec` (`4.03`).
  - The 6 red bars are `Jul` through `Dec`: every month in the second half of the year is below average.
  - Business meaning: spending rose faster than revenue. Revenue doubled, but each marketing dollar brought in about 32% less revenue in December than in January, and the cost per new customer rose from `48.5` to `68.0`. The CMO should ask whether the November and December campaigns were worth their extra cost.